ECS7020P — ML Exam Cheatsheet

1. Quick Reference

Imports

```
import numpy as np
import matplotlib.pyplot as plt
```

Notation

Symbol	Meaning
N	Number of samples
K	Number of predictors
D	Polynomial degree
x_i	Predictor(s) of sample i
y_i	True label of sample i
$\hat{y}_i$	Predicted label: $\hat{y}_i = f(x_i)$
e_i	Error: $e_i = y_i - \hat{y}_i$
X	Design matrix ($N \times (K+1)$, first col = 1)
y	Label vector ($N \times 1$)
w	Coefficient vector
η	Learning rate (gradient descent)

Variable Conventions

```
X = np.array(...)   # Design matrix (N x K+1), first column of ones
y = np.array(...)   # Label vector (N x 1)
w = np.array(...)   # Coefficients vector
y_hat = X @ w       # Predictions
e = y - y_hat       # Error vector
```

---

2. Regression

Error Metrics

Mean Squared Error (MSE) — average of squared errors:

$$E_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

```
mse = np.mean((y - y_hat)**2)
```

Mean Absolute Error (MAE) — average of absolute errors:

$$E_{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

```
mae = np.mean(np.abs(y - y_hat))
```

Root Mean Squared Error (RMSE) — std dev of prediction error:

$$E_{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}$$

```
rmse = np.sqrt(np.mean((y - y_hat)**2))
```

R-squared — proportion of variance explained:

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

```
r2 = 1 - np.sum((y - y_hat)**2) / np.sum((y - np.mean(y))**2)
```

Simple & Multiple Linear Regression

Model: $f(\mathbf{x}_i) = \mathbf{w}^T\mathbf{x}_i = w_0 + w_1 x_{i,1} + \cdots + w_K x_{i,K}$

In matrix form: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$

Normal Equations (Least Squares / MMSE Solution)

The coefficients that minimise MSE on the training data:

$$\mathbf{w}_{best} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

```
# Build design matrix (prepend column of ones)
X = np.column_stack([np.ones(N), x])  # simple linear
w = np.linalg.inv(X.T @ X) @ X.T @ y
y_hat = X @ w
```

Worked example (4 samples, simple linear):

```
x = np.array([2, 4, 1, 3])
y = np.array([1, 5, 2, 2])
X = np.column_stack([np.ones(4), x])       # 4x2 design matrix
w = np.linalg.inv(X.T @ X) @ X.T @ y      # [w0, w1]
# XTX_inv = [[1.5, -0.5], [-0.5, 0.2]]    (given in exam)
```

Polynomial Regression

Model: $f(x_i) = w_0 + w_1 x_i + w_2 x_i^2 + \cdots + w_D x_i^D$

Key insight: Treat each power as a separate predictor → same normal equations.

```
# Design matrix for polynomial degree D
X = np.column_stack([x**d for d in range(D+1)])  # N x (D+1)
w = np.linalg.inv(X.T @ X) @ X.T @ y
```

Important: If $D + 1 \geq N$ (more coefficients than samples), the model can achieve MSE = 0 on training data → likely overfitting. $\mathbf{X}^T\mathbf{X}$ may become singular.

Gradient Descent

Update rule for MSE minimisation:

$$\mathbf{w}_{new} = \mathbf{w}_{old} - \eta \nabla E_{MSE}(\mathbf{w}_{old})$$

MSE gradient for linear regression:

$$\nabla E_{MSE}(\mathbf{w}) = \frac{-2}{N}\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

```
w = np.zeros(X.shape[1])           # initial weights
eta = 0.01                          # learning rate
for i in range(1000):               # num iterations
    grad = (-2/N) * X.T @ (y - X @ w)
    w = w - eta * grad
```

Step size: Too small → slow convergence. Too large → overshooting.
Stopping: Fixed iterations, error threshold, or relative change.
Local vs global: Convex surfaces (like MSE for linear) have one minimum. Non-convex surfaces can trap GD in local optima → restart from multiple initialisations.

Plotting a Regression Model

```
x_plot = np.linspace(x.min(), x.max(), 100)
X_plot = np.column_stack([x_plot**d for d in range(D+1)])
plt.scatter(x, y, label='Data')
plt.plot(x_plot, X_plot @ w, 'r-', label='Model')
plt.xlabel('x'); plt.ylabel('y'); plt.legend(); plt.show()
```

---

3. Classification

Problem Setup

- Label y is **discrete** (classes), not continuous
- Model partitions predictor space into **decision regions** separated by **decision boundaries**
- Binary: two classes (e.g., ∘ and ×)

Linear Classifiers

Boundary defined by: $\mathbf{w}^T\mathbf{x} = 0$

where $\mathbf{x} = [1, x_1, x_2, ...]^T$ (extended vector with 1 prepended).

Classification rule: - $\mathbf{w}^T\mathbf{x}_i > 0$ → class ∘ (positive) - $\mathbf{w}^T\mathbf{x}_i < 0$ → class × (negative) - $\mathbf{w}^T\mathbf{x}_i = 0$ → on the boundary

Example: $w = [2, 1, 0]^T$ → boundary is $2 + x_1 = 0 \rightarrow x_1 = -2$.

Accuracy & Error Rate

$$\hat{A} = \frac{\text{correct predictions}}{N}, \quad \hat{E} = 1 - \hat{A}$$

```
accuracy = np.mean(y_pred == y_true)
error_rate = 1 - accuracy
```

Confusion Matrix (Binary)

	Predicted ∘	Predicted ×
Actual ∘	True Positive (TP)	False Negative (FN)

	Predicted ∘	Predicted ×
Actual ×	False Positive (FP)	True Negative (TN)

$$\text{Sensitivity (Recall, TPR)} = \frac{TP}{TP + FN}$$

$$\text{Specificity (TNR)} = \frac{TN}{TN + FP}$$

$$\text{Precision (PPV)} = \frac{TP}{TP + FP}$$

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

```
TP = np.sum((y_pred == 1) & (y_true == 1))
FN = np.sum((y_pred == 0) & (y_true == 1))
FP = np.sum((y_pred == 1) & (y_true == 0))
TN = np.sum((y_pred == 0) & (y_true == 0))
sensitivity = TP / (TP + FN)      # aka recall, TPR
specificity = TN / (TN + FP)      # aka TNR
precision = TP / (TP + FP)        # aka PPV
f1 = 2 * precision * sensitivity / (precision + sensitivity)
```

Confusion matrix as rates — divide each column by its total. Diagonal = per-class accuracies. Useful for **imbalanced datasets** where raw counts are misleading.

Common metric pairs (improving one usually hurts the other): - Sensitivity vs Specificity - Precision vs Recall

Logistic Regression

Logistic (sigmoid) function — maps distance to boundary → certainty:

$$p(d) = \frac{e^d}{1 + e^d} = \frac{1}{1 + e^{-d}}$$

where $d = \mathbf{w}^T \mathbf{x}_i$.

Properties: $p(0) = 0.5$, $p(\infty) = 1$, $p(-\infty) = 0$.

- $p(\mathbf{x}_i)$ = certainty that $y_i = \circ$
- $1 - p(\mathbf{x}_i)$ = certainty that $y_i = \times$

```
def sigmoid(d):
    return 1 / (1 + np.exp(-d))
```

Likelihood — how well coefficients **w** explain the labelled data:

$$L = \prod_{y_i = \times} (1 - p(\mathbf{x}_i)) \prod_{y_i = \circ} p(\mathbf{x}_i)$$

Log-likelihood (easier to work with):

$$\ell = \sum_{y_i=\times} \log(1 - p(\mathbf{x}_i)) + \sum_{y_i=\circ} \log(p(\mathbf{x}_i))$$

Equivalently with $y_i \in \{0, 1\}$:

$$\ell = \sum_{i=1}^{N} [y_i \log p(\mathbf{x}_i) + (1 - y_i) \log(1 - p(\mathbf{x}_i))]$$

Training: Maximise ℓ using gradient descent (no closed-form solution).

```
w = np.zeros(X.shape[1])
eta = 0.01
for _ in range(1000):
    p = sigmoid(X @ w)
    grad = X.T @ (y - p)        # gradient of log-likelihood
    w = w + eta * grad           # ascent (maximising)
```

k-Nearest Neighbours (kNN)

Non-parametric: no explicit boundary, stores entire training set.

Algorithm: For a new sample **x**: 1. Compute distance to all training samples 2. Find K closest neighbours 3. Assign most common label among neighbours

```
def knn_predict(X_train, y_train, x_new, K):
    dists = np.sqrt(np.sum((X_train - x_new)**2, axis=1))
    idx = np.argsort(dists)[:K]
    labels, counts = np.unique(y_train[idx], return_counts=True)
    return labels[np.argmax(counts)]
```

K controls complexity: $K = 1$ → complex boundaries (overfitting risk). Large K → smoother boundaries (underfitting risk).

Bayes Classifier

Assign label of the class with highest posterior:

$$P(C_k|\mathbf{x}) \propto P(\mathbf{x}|C_k) \cdot P(C_k)$$

- $P(C_k)$ = **prior** (proportion of class k in data)
- $P(\mathbf{x}|C_k)$ = **likelihood** / class-conditional density
- $P(C_k|\mathbf{x})$ = **posterior** — what we actually want

Don't confuse likelihood with posterior! Likelihood = $P(\mathbf{x}|C_k)$ (probability of data given class). Posterior = $P(C_k|\mathbf{x})$ (probability of class given data).

Odds ratio formulation (binary, classes $\circ$ and $\times$):

$$\frac{P(y = \circ \mid \mathbf{x})}{P(y = \times \mid \mathbf{x})} = \frac{p(\mathbf{x} \mid y = \circ)\, P(y = \circ)}{p(\mathbf{x} \mid y = \times)\, P(y = \times)} \gtrless 1$$

If ratio > 1 → classify as $\circ$. If < 1 → classify as $\times$.

The Bayes classifier achieves the **highest possible accuracy**. No other classifier can beat it (it uses true posteriors).

Estimating priors from data:

$$\hat{P}(C_k) = \frac{\text{number of class } k \text{ samples}}{N}$$

```
prior_k = np.sum(y == k) / len(y)
```

Key exam insight: If two classes have equal variance and a sample is equidistant from both means, classify according to the **higher prior**.

Discriminant Analysis (LDA & QDA)

Assumes class-conditional densities are **Gaussian**. Estimate parameters from data, then use Bayes rule.

One predictor — estimate per-class mean and variance:

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{y_i = k} x_i, \quad \hat{\sigma}_k^2 = \frac{1}{N_k - 1} \sum_{y_i = k} (x_i - \hat{\mu}_k)^2$$

```
mu_k = np.mean(x[y == k])
var_k = np.var(x[y == k], ddof=1)   # unbiased (N-1 denominator)
```

Multiple predictors — estimate mean vector μ_k and covariance matrix Σ_k:

```
X_k = X[y == k]                     # samples of class k (N_k x D)
mu_k = np.mean(X_k, axis=0)         # (D,)
Sigma_k = np.cov(X_k.T)             # (D x D), unbiased by default
```

LDA vs QDA: - **LDA** (Linear Discriminant Analysis): assumes $\Sigma_\circ = \Sigma_\times$ → **linear** boundary - **QDA** (Quadratic Discriminant Analysis): different covariance matrices → **quadratic** boundary

Naive Bayes assumption: predictors are independent → covariance matrix is diagonal:

$$\Sigma_k = \text{diag}(\sigma_{k,1}^2, \sigma_{k,2}^2, \ldots)$$

This simplifies computation — just estimate each feature's variance separately.

Gaussian density (1D) for classification:

$$p(x \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right)$$

```
def gauss_pdf(x, mu, sigma):
    return (1 / (np.sqrt(2 * np.pi) * sigma)) * np.exp(-0.5 * ((x - mu) / sigma)**2)

# Classify by comparing posterior odds
odds = (gauss_pdf(x_new, mu_A, sig_A) * prior_A) / (gauss_pdf(x_new, mu_B, sig_B) * prior_B)
y_pred = 'A' if odds > 1 else 'B'
```

Misclassification Cost & Calibration

Accuracy treats all errors equally. Often they're not — misdiagnosing a sick patient ≠ misdiagnosing a healthy one.

Cost-sensitive decision rule:

$$\frac{C_\times \cdot P(y = \circ \mid \mathbf{x})}{C_\circ \cdot P(y = \times \mid \mathbf{x})} \gtrless 1 \quad \Longleftrightarrow \quad \frac{P(y = \circ \mid \mathbf{x})}{P(y = \times \mid \mathbf{x})} \gtrless \frac{C_\circ}{C_\times}$$

where $C_\circ$ = cost of misclassifying a $\circ$ sample, $C_\times$ = cost of misclassifying a $\times$ sample.

General threshold form:

$$\frac{P(y = \circ \mid \mathbf{x})}{P(y = \times \mid \mathbf{x})} \gtrless T$$

- $T = 1$: standard Bayes (maximise accuracy)
- $T = C_\circ / C_\times$: minimise expected cost
- Changing T = **calibration** — shifts decision boundary without retraining

Per-Class Accuracies

$$\hat{A}_\circ = \frac{\text{true } \circ \text{ predictions}}{\text{total } \circ \text{ samples}}, \quad \hat{A}_\times = \frac{\text{true } \times \text{ predictions}}{\text{total } \times \text{ samples}}$$

Overall accuracy $\hat{A}$ hides how each class is treated. Per-class accuracies reveal it. In general, **maximising one deteriorates the other** — there's a trade-off.

ROC Curve & AUC

The **ROC plane** plots sensitivity (TPR) vs 1−specificity (FPR) as the threshold T varies from 0 to ∞.

- **Top-left corner** = perfect classifier (sensitivity = 1, FPR = 0)
- **Diagonal** = random classifier
- Each trained model has its own ROC curve (from varying T)

```
# Manual ROC curve (given thresholds and predictions)
thresholds = [0.5, 1.5, 3.5, 5.5, 7.5, 9.5]
sensitivities = []
fprs = []
for t in thresholds:
    y_pred = (scores >= t).astype(int)
    tp = np.sum((y_pred == 1) & (y_true == 1))
    fn = np.sum((y_pred == 0) & (y_true == 1))
    fp = np.sum((y_pred == 1) & (y_true == 0))
    tn = np.sum((y_pred == 0) & (y_true == 0))
    sensitivities.append(tp / (tp + fn))
    fprs.append(fp / (fp + tn))

plt.plot(fprs, sensitivities, 'o-')
plt.xlabel('1 - Specificity (FPR)')
plt.ylabel('Sensitivity (TPR)')
plt.title('ROC Curve')
plt.show()
```

AUC (Area Under the ROC Curve): - AUC ≈ 1.0 → excellent classifier - AUC ≈ 0.5 → no better than random - AUC < 0.5 → worse than random (flip predictions!)

Use case: Compare two model families by their AUC — higher AUC = better overall discrimination regardless of threshold choice.

Classifier Comparison Summary

Property	Logistic Regression	LDA	QDA	kNN
Boundary shape	Linear	Linear	Quadratic	Any shape
Stability (small N)	Unstable	Stable	Stable	Depends on K

Property	Logistic Regression	LDA	QDA	kNN
Outlier robustness	Robust	Sensitive	Sensitive	Depends on K
Multiclass	Needs extensions	Natural	Natural	Natural
Prior knowledge	Hard to incorporate	Easy (Bayesian)	Easy (Bayesian)	N/A

---

4. Model Evaluation & Methodology

Train / Test Split

- **Training set:** Used to fit/tune model parameters
- **Test set:** Used ONCE to estimate deployment performance — never used during training

Critical rule: Never use test data for training. The test set must remain unseen.

```
# Manual split
N = len(y)
idx = np.random.permutation(N)
split = int(0.7 * N)
X_train, X_test = X[idx[:split]], X[idx[split:]]
y_train, y_test = y[idx[:split]], y[idx[split:]]
```

Randomisation matters: Always shuffle before splitting to avoid structural bias in the data.

Split Ratio Trade-off

More training data (F → 1)	More test data (F → 0)
Better model (less overfitting)	Better estimate of deployment MSE
Poor test estimate (few test samples)	Worse model (trained on less data)

Validation (Hyperparameter Selection)

Problem: How to choose between model families (e.g., polynomial degree D)?

Solution: Split available data into training + validation: 1. Train each candidate model on training subset 2. Evaluate each on validation subset 3. Select the family with lowest validation error 4. Retrain selected family on ALL available data (train + validation) 5. Final model tested on held-out test set

K-Fold Cross-Validation

Splits data into K folds. Each fold takes a turn as validation set.

1. Divide data into K equal parts
2. For each fold k: train on remaining $K-1$ folds, validate on fold k
3. Average the K validation scores

Leave-One-Out (LOO): $K = N$. Most thorough but expensive.

Overfitting vs Underfitting

	Training Error	Deployment Error	Cause
Underfitting	High	High	Model too simple
Overfitting	Low	High	Model too complex / too little data

	Training Error	Deployment Error	Cause
Just right	Low	Low	Good balance

Key: You CANNOT detect overfitting from training error alone. You need to compare training vs deployment/test error.

Bias-Variance Trade-off

- **Bias:** Error from wrong model assumptions (underfitting). A linear model on quadratic data has high bias.
- **Variance:** Error from sensitivity to training data fluctuations (overfitting). Complex models have high variance.
- **Noise:** Irreducible error from randomness in the data.

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

As complexity ↑: bias ↓, variance ↑. Sweet spot minimises total error.

---

5. Maths Foundations

Vectors

$$\mathbf{x} = [x_1, x_2, \ldots, x_K]^T$$

Dot product: $\mathbf{a}^T\mathbf{b} = \sum_i a_i b_i$ — scalar result

Euclidean distance: $d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_i (a_i - b_i)^2}$

```
dot = a @ b                          # or np.dot(a, b)
dist = np.sqrt(np.sum((a - b)**2))  # or np.linalg.norm(a - b)
```

Matrices

Transpose: $(A^T)_{ij} = A_{ji}$

Multiplication: $(\mathbf{AB})_{ij} = \sum_k A_{ik} B_{kj}$ — inner dimensions must match: $(m \times n)(n \times p) = (m \times p)$

Inverse: $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ — exists only if $\mathbf{A}$ is square and non-singular

```
A_T = A.T
AB = A @ B
A_inv = np.linalg.inv(A)
```

Key Matrix Identity (used in normal equations)

$\mathbf{X}^T\mathbf{X}$ is $(K+1) \times (K+1)$

$(\mathbf{X}^T\mathbf{X})^{-1}$ exists when columns of $\mathbf{X}$ are linearly independent

Derivatives (Gradient)

Gradient of scalar function w.r.t. vector **w**:

$$\nabla_{\mathbf{w}} f = \left[\frac{\partial f}{\partial w_0}, \frac{\partial f}{\partial w_1}, ...\right]^T$$

Key result for MSE:

$$\nabla_{\mathbf{w}} \frac{1}{N}(\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) = \frac{-2}{N}\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w})$$

Setting to zero → normal equations.

Probability Basics

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

- **Prior:** $P(A)$ — belief before seeing evidence
- **Likelihood:** $P(B|A)$ — probability of evidence given hypothesis
- **Posterior:** $P(A|B)$ — updated belief after evidence

Used in: Bayes classifier, logistic regression interpretation.

---

6. Exam Patterns & Tips

Common Question Types

Q1: Regression (manual calculation) - Given small dataset → build design matrix **X**, compute $(\mathbf{X}^T\mathbf{X})^{-1}$ (often given), find **w** - Calculate training MSE by hand - Discuss: would a higher-degree polynomial have lower training MSE? (Yes, but risk of overfitting)

Q2: Classification - Given linear classifier **w** → identify decision boundary and regions - Build confusion matrix from dataset + boundary - Calculate sensitivity and specificity - Bayes classifier: compute priors, use class means, classify new sample

Q3: Methodology / Conceptual - Explain local vs global minima (error surface) - Describe k-means convergence to local minimum - Design strategy to improve (multiple initialisations) - Cross-validation explanation - Overfitting vs underfitting identification

Approach Template: Regression by Hand

1. Write out **X** (column of 1s + predictor columns)
2. Compute $\mathbf{X}^T\mathbf{X}$ and $\mathbf{X}^T\mathbf{y}$
3. If $(\mathbf{X}^T\mathbf{X})^{-1}$ is given, multiply: $\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$
4. Compute predictions: $\hat{y}_i = \mathbf{w}^T\mathbf{x}_i$
5. Compute errors: $e_i = y_i - \hat{y}_i$
6. MSE = mean of e_i^2

Approach Template: Confusion Matrix

1. For each sample, compute $\mathbf{w}^T\mathbf{x}_i$
2. Classify: positive if > 0, negative if < 0
3. Compare with true labels → fill TP, FP, TN, FN
4. Sensitivity = TP/(TP+FN), Specificity = TN/(TN+FP)

Q4: Discriminant Analysis / Bayes (Week 5 style) - Estimate class means and variances from data (use unbiased variance: divide by $N_k - 1$) - Compute Gaussian densities, multiply by priors, compare odds ratio - If priors change → threshold changes, **no retraining** needed - Sketch ROC curve: compute sensitivity & specificity for multiple thresholds, plot (1−spec, sens)

Approach Template: Discriminant Analysis by Hand

1. Compute per-class mean: $\hat{\mu}_k = \frac{1}{N_k} \sum x_i$
2. Compute per-class variance (unbiased): $\hat{\sigma}_k^2 = \frac{1}{N_k - 1} \sum (x_i - \hat{\mu}_k)^2$
3. For new sample x^*: compute $p(x^*|C_k)$ using Gaussian formula for each class
4. Multiply by prior: $p(x^*|C_k) \cdot P(C_k)$
5. Compare → assign to class with highest value (or use odds ratio $\gtrless T$)

Approach Template: ROC Curve by Hand

1. For each threshold T, classify all samples
2. Count TP, FP, TN, FN → compute sensitivity & specificity
3. Plot points $(1 - \text{specificity}, \text{sensitivity})$
4. Connect points → ROC curve
5. Better curve = closer to top-left corner; AUC closer to 1

Common Pitfalls

- **Using training MSE to compare models of different complexity** → WRONG. Must use test/validation error.
- **Polynomial degree ≥ N** → can get MSE = 0 on training, but model is useless. $\mathbf{X}^T\mathbf{X}$ may be singular.
- **Forgetting the column of ones** in the design matrix → wrong w_0.
- **Confusing sensitivity/specificity** → sensitivity = recall = TP rate (about the positive class). Specificity = TN rate (about the negative class).
- **Bayes classifier with equal variances** → when sample is equidistant from both class means, classify by **prior** (higher prior wins).
- **Gradient descent direction** → for minimising cost: subtract gradient. For maximising likelihood: add gradient.
- **Test set must never be used during training/validation.** Validation data can be reused for final training after model selection.